TARACRAFT / PRODUKTDOKUMENTATION



Das Handbuch

Verbindungen einrichten. Aufträge verstehen.
Den Dienst sicher betreiben.

VERBINDUNG & BETRIEB

DATEN & WIEDERHERSTELLUNG

API & FEHLERHILFE

Dokumentationsstand 17. Oktober 2026

Quellfunktionen dokumentiert. Vollständige Betriebsabnahmen bleiben offen.

Markenillustration: mit KI aus der bereitgestellten TaraCraft-Vorlage bearbeitet.

Dein Weg durchs Handbuch

Vom ersten Überblick bis zur gezielten Fehlerhilfe. Die Kapitel und Seitenzahlen sind anklickbar; technische Vertiefungen führen zur aktuellen Online-Dokumentation.

1. Was der Server übernimmt 4

2. Den passenden Einrichtungsweg wählen 5

3. Voraussetzungen und Konfigurationsdaten 6

4. Quellen prüfen und einen Kandidaten bauen 7

5. Daten und Compose gemeinsam vorbereiten 9

6. Engines und Profile passend zusammenführen 10

7. Home Assistant verbinden 12

8. Einen Slicing-Ablauf nachvollziehbar durchführen 13

9. Authentifizierung, TLS und Datengrenzen 14

10. Status beobachten und Fehler früh einordnen 15

11. Wartung, Sicherung und Wiederherstellung 16

12. Fehlersuche: Symptom, Prüfung, Maßnahme 18

13. Häufige Fragen 20

14. Begriffe und nächste Orientierung 22

Orientierung

ARCHITEKTUR AUF EINEN BLICK



Slicer-Zugangsdaten bleiben serverseitig. Der Browser erhaelt keine Secrets.

Slicing, Ergebnispruefung und Druckfreigabe sind getrennte Schritte.

Schema zur Orientierung; keine Oberflaechen- oder Betriebsabnahme.

So liest du den Entwicklungsstand

Die dokumentierten Funktionen sind im Quellstand vorhanden. Daraus folgt keine vollständige Geräte-, Installations- oder Betriebsabnahme. Stand: 77 Funktionen und 41 Zusatzanforderungen; vollständige Abnahmen bleiben offen. Ein grüner Testlauf oder eine veröffentlichte Dokumentation ist keine Gesamtfreigabe.

Für Anwender

Beginne mit dem Überblick und dem sicheren Einstieg. Die nummerierten Abläufe erklären Voraussetzungen, Bedienung und erwartetes Ergebnis. Die Fehlerhilfe hilft beim nächsten sinnvollen Schritt.

Für Administratoren

Lies zuerst Einrichtung, Verbindungswege und Datenablage. Halte Sicherung, Update und Wiederherstellung zusammen. Die separate Linux-Neuinstallation bleibt der letzte Verifikationstest.

Für Entwickler

Nutze die verlinkten Quellverträge und technischen Referenzen. Quellprüfung, Paketfreigabe, Installation und tatsächlicher Betrieb sind getrennte Nachweise.

1. Was der Server übernimmt

Ein Slicing-Auftrag umfasst mehr als eine Modelldatei. Die ausgewählte Engine braucht passende Maschinen-, Material- und Prozessdaten, vollständige Ressourcen und einen Ort für Zwischendateien und Ergebnisse. Der Server hält diese Bestandteile auseinander. Dadurch lässt sich bei einem Problem gezielter feststellen, ob die Verbindung, der Auftrag, die Profile oder die Engine betroffen sind.

Die HTTP-API nimmt Anfragen entgegen und verwaltet Aufträge. Der Worker verarbeitet diese Aufträge getrennt von der API. Im Compose-Kandidaten läuft der Worker ohne eigenes Netzwerk. Die Engine-Dateien und die gemeinsam verwendeten Profile sind für die laufenden Verbraucher nur lesbar eingebunden. Modelle, Auftragshistorie und Ergebnisse liegen außerhalb des Images.

Home Assistant stellt die Integration und den Verbindungskontext bereit. Der Browser arbeitet über die Home-Assistant-API und erhält keine dauerhaften Server-Secrets. Ultimate 3D Studio ist damit die Bedienebene; Ultimate Slicer Server ist der verarbeitende Dienst. Ein laufender Server ersetzt weder die Studio-Integration noch eine korrekt eingerichtete Druckerverbindung.

Bestandteil	Aufgabe	Was daraus allein noch nicht folgt
Studio in Home Assistant	Modelle und Slicing-Vorbereitung bedienen	Eine betriebsbereite Engine ist vorhanden
HTTP-API	Verbindung, Informationen und Aufträge verwalten	Ein erfolgreicher Auftrag wurde berechnet
Worker	Aufträge unter den vorgesehenen Laufzeitbedingungen verarbeiten	Alle Profile passen zum gewählten Drucker
Engine	Die eigentliche Slicing-Berechnung durchführen	Jede externe Ressource und jedes Plugin ist geprüft
Persistenter Speicher	Konfiguration, Profile, Jobs und Ergebnisse erhalten	Eine Sicherung wurde erfolgreich wiederhergestellt

Für die Einordnung eines Status ist diese Trennung entscheidend. Eine erreichbare API kann einen fehlenden Enginebestand melden. Ein gesunder Worker kann kontrolliert pausieren. Eine vorhandene ausführbare Engine-Datei bestätigt noch keinen vollständigen Profilbestand. Das Handbuch führt deshalb Verbindung, Einrichtung und Slicing als getrennte Schritte.

2. Den passenden Einrichtungsweg wählen

Die Integration kennt unterschiedliche Wege. Wähle den Weg nach deiner vorhandenen Umgebung, nicht nach einem Fehler, den du gerade umgehen möchtest. Ein fehlendes Supervisor-Paket wird beispielsweise nicht dadurch verfügbar, dass du stattdessen den Hostinstaller auswählst.

Weg	Voraussetzung	Gegenwärtige Bedeutung
Externer Slicer	Ein eigenständig eingerichteter kompatibler Dienst mit HTTPS und Secret	Home Assistant verbindet sich mit diesem Dienst
Supervisor-App	Home Assistant mit Supervisor und passende App-Paketreferenz	Verwaltungslogik vorhanden; Installation ohne freigegebene Paketreferenz gesperrt
Provisionierter Unix-Host	Bereits ausdrücklich eingerichteter, vertrauenswürdiger Installationskanal	Eigener Hostvertrag; kein automatischer Ersatz für Supervisor
Quellprüfung und Compose-Kandidat	Vollständiger Klon, Buildwerkzeuge und isolierter Linux-Testpfad	Entwicklungs- und Vorprüfungsweg, keine veröffentlichte Komplettinstallation

Ein externer Dienst ist sinnvoll, wenn du den Linux-Server unabhängig von Home Assistant verwaltest. Home Assistant benötigt dessen URL, das API-Secret und ein passendes Zertifikatsvertrauen. Der Dienst muss tatsächlich eingerichtet sein; ein Eintrag in einem Formular legt noch keine Engines oder Datenverzeichnisse an.

Die Supervisor-Verwaltung verwendet den von Home Assistant bereitgestellten Supervisor-Client. Zusätzliche Core-Mounts, eine allgemeine Shell in Home Assistant oder ein Docker-Socket sind für diesen Weg nicht erforderlich. Ohne Supervisor bleiben Studio und der externe Verbindungsweg grundsätzlich getrennt nutzbar. Eine freigegebene App-Paketreferenz ist im geprüften Stand noch nicht hinterlegt.

Der provisionierte Hostweg setzt eine vorbereitete Vertrauensbeziehung und einen begrenzten Installationskanal voraus. Er ist für die dafür eingerichtete Umgebung vorgesehen. Frei eingegebene Shellbefehle gehören nicht zu diesem Bedienweg. Seine technischen Details beschreiben Hostkopplung, Transport und Lifecycle.

Das native Linux-Paket besitzt einen eigenen Installer und systemd-Vertrag. Es wird nicht durch HACS installiert. Der veröffentlichte Paketstand trägt `native_deploy_ready=false`; damit sind Installation und Upgrade in diesem Entwicklungsstand gesperrt. HACS betrifft die Home-Assistant-Integration, nicht die Beschaffung einer vollständigen nativen Engine-Laufzeit.

3. Voraussetzungen und Konfigurationsdaten

Für die Dateiprüfung nennt die vorhandene Installationsanleitung Python 3.11 oder neuer. Für den Compose-Kandidaten werden Linux/amd64, Docker mit Compose und ein vollständiger Quellbaum vorausgesetzt. Andere Architekturen sind nicht allgemein zugesagt. Ein vorhandener ARM-Rechner wird durch einen erfolgreichen Python-Test nicht automatisch zu einer unterstützten Engineplattform.

Belastbare Mindestwerte für RAM, CPU-Leistung, freien Laufzeitspeicher und typische Slicing-Dauer sind in den geprüften Endnutzerunterlagen noch nicht festgelegt. Diese Werte hängen auch von Modell, Engine und Ressourcenbestand ab. Plane deshalb anhand des tatsächlich bereitgestellten Pakets und späterer Messungen. Eine geschätzte Zahl wäre hier keine verlässliche Systemanforderung.

Vor der Einrichtung sollten folgende Daten vorliegen:

Angabe	Bedeutung	Wo sie hingehört
Server-URL	Tatsächlich erreichbare HTTPS-Adresse des Slicers	Geschützte Verbindungseinstellungen in Home Assistant
API-Secret	Authentifizierung gegenüber dem Dienst	Private Secret-Datei beziehungsweise geschützter HA-Eintrag
Zertifikatsname und CA-Vertrauen	Identität des Servers	TLS-Konfiguration und vertrauender Client
Image-ID oder späterer Digest	Exakter Anwendungsstand	Betriebsnachweis und Konfiguration
Enginegeneration	Zusammengehöriger Engine- und Ressourcenbestand	Geprüfter Engine-Speicher
Daten- und Profilpfad	Dauerhafte Ablage für den jeweiligen Dienst	Hostkonfiguration, außerhalb des Repositorys
Benutzer- und Gruppenrechte	Zugriff der vorgesehenen Laufzeitidentität	Geprüfter Rechtevertrag auf dem Host

Das Secret ist ein Zugangsschlüssel, kein bequemes Diagnosefeld. Für den Compose-Kandidaten ist eine lokal erzeugte Datei mit mindestens 32 druckbaren ASCII-Zeichen dokumentiert. Der konkrete Inhalt gehört weder in Git noch in eine öffentliche Umgebungsdatei, einen Screenshot oder einen Fehlerbericht. Eine Anleitung muss nur den Speicherort und die Anforderungen erklären.

Ein Build-Ausgabeverzeichnis muss neu sein und außerhalb des gesamten Klons liegen. Laufzeitkonfiguration, Modelle und Schlüssel sind ebenfalls keine Build-Eingaben. Wer den Entwicklungsordner mit beliebigen privaten Dateien als Docker-Kontext verwendet, verlässt den vorgesehenen Dateiauswahlvertrag. Der Kontextgenerator übernimmt stattdessen eine festgelegte Auswahl benötigter Quellen.

4. Quellen prüfen und einen Kandidaten bauen

Dieser Abschnitt ist ein dokumentierter Entwicklungsweg. Die Befehle stammen aus Docker-Schnellstart, Installation und Laufzeitkonfiguration. Sie prüfen Quellen oder erzeugen einen Kandidaten. Ein Containerbau oder eine Paketprüfung ersetzt keine Betriebsabnahme und ist nicht mit der abschließenden Linux-Neuinstallation gleichzusetzen.

Alle folgenden relativen Pfade gelten ab der Wurzel des eigenständigen Slicer-Repositories. Im Studio-Gesamtrepository ist `studio/` die entsprechende Komponentenwurzel. Prüfe diese Zuordnung, bevor du einen Befehl übernimmst; derselbe relative Pfad kann aus dem falschen Arbeitsverzeichnis auf etwas anderes zeigen.

Quellen und Profilherkunft prüfen

```
python3 Slicer-Server/build_repository.py --verify .
python3 Slicer-Server/verify_profile_sources.py --source-root .
```

Der erste Befehl gehört zum eigenständigen Slicer-Repository und kontrolliert dessen Lieferung. Die Profilprüfung ist auch in der vollständigen Studio-Quellzusammenstellung vorhanden. Sie verifiziert die beigefügten Originaldaten und Zuordnungen offline. Sie lädt keine Engine herunter und führt weder Profilcode noch G-Code aus.

Wenn eine Prüfung eine abweichende Datei meldet, ersetze nicht einfach den erwarteten Hash. Kläre, ob du einen anderen Quellstand, eine unvollständige Kopie oder eine tatsächliche Änderung vorliegen hast. Ein Manifest soll Abweichungen sichtbar machen; es ist kein Formular, dessen Werte an jedes gewünschte Ergebnis angepasst werden.

Einen begrenzten Build-Kontext erzeugen

```
SLICER_BUILD_ROOT="$(mktemp -d /tmp/ultimate-slicer-build-XXXXXX)"
python3 Slicer-Server/prepare_context.py --source-root "$PWD" \
  --output "$SLICER_BUILD_ROOT/context"
```

Der neue Kontext enthält API, Worker, benötigte Helfer, Metadaten, Provenienz, Profilquellen und Lizenztexte. Er enthält keine Engine-Binaries, privaten TLS-Schlüssel, API-Token, Aufträge oder Modelle. Ein bereits vorhandenes Ziel wird nicht ungefragt geleert. Für einen erneuten Versuch ist ein neuer eigener Kontext vorgesehen.

Das Basisimage bewusst festlegen

```
export PYTHON_BASE='GEPRUEFTE_REFERENZ@sha256:GEPRUEFTER_DIGEST'
docker build --pull=false --build-arg PYTHON_BASE="$PYTHON_BASE" \
  --tag ultimate-slicer-server:candidate "$SLICER_BUILD_ROOT/context"
docker image inspect ultimate-slicer-server:candidate
```

Die Referenz im Beispiel ist ausdrücklich ein Platzhalter. Ohne einen tatsächlichen geprüften Wert ist dieser Block nicht ausführbar. Ein fester Base-Image-Digest bindet außerdem nicht automatisch jede beim Build verwendete Distributionsabhängigkeit. Zum späteren Release gehören daher auch deren konkrete Versionen, Architektur und Lizenzhinweise.

Das Prüfimage richtig verstehen

```
docker run --rm --network none --read-only --user 65532:65532 \
  --cap-drop ALL --security-opt no-new-privileges:true \
  ultimate-slicer-server:candidate check
```

Dieser dokumentierte Lauf prüft das Paket in einer eingeschränkten Containerumgebung. Er bindet keine vorhandenen HA-Daten und keine Drucker ein. Ein erfolgreiches Ergebnis bestätigt den geprüften Umfang, nicht die Verfügbarkeit einer Slicing-Engine. Das Standardkommando des Images bleibt die Paketprüfung; der Dienstbetrieb wird eigens über Compose eingerichtet.

5. Daten und Compose gemeinsam vorbereiten

Ein Container ist austauschbar, deine Profile und Ergebnisse sind es nicht. Der Compose-Kandidat legt dafür getrennte Bereiche fest. Bereite sie unter dem vorgesehenen Rechtevertrag vor, bevor du Dienste startest. Fehlende Bind-Mounts werden nicht still als leere Hostverzeichnisse erzeugt.

Variable	Erwarteter Inhalt	Wichtige Eigenschaft
`SLICER_IMAGE`	Exakte lokal vorhandene Image-ID	`pull_policy: never`
`SLICER_DATA`	Neues persistentes Datenverzeichnis	Vorgesehene Runtime-Rechte
`SLICER_ENGINE`	Gesamter Engine-Stamm	Für API und Worker nur lesbar
`SLICER_PROFILES`	Bereits vorbereiteter Profilstamm	Stammordner, nicht nur `printers/`
`SLICER_TOKEN`	Private Datei mit dem API-Secret	Kein Secret-Text im öffentlichen Repository
`SLICER_TLS`	Verzeichnis mit `server.crt` und `server.key`	Name, Vertrauen und Dateirechte müssen passen
`SLICER_BIND`	Bewusst ausgewählte Bind-Adresse	Standardmäßig Loopback
`SLICER_PORT`	API-Port auf dem Host	Dokumentierter Compose-Standard: 8099
`SLICER_BOOTSTRAP_REQUEST_ID`	Eigene Initialisierungs-/Wartungskennung	Nur dem eigenen Vorgang zuordnen

Der dokumentierte Compose-Pfad verwendet für API und Worker UID/GID 65532. Die Hostinitialisierung vergibt Rechte gezielt an vorbereitete Daten- und Profilbereiche. Daraus folgt keine Empfehlung, ganze vorhandene Verzeichnisse rekursiv umzuschreiben. Rootless Docker und User-Namespace-Remapping benötigen einen eigenen passenden Rechtevertrag.

Compose trennt Dateninitialisierung, Profilinitialisierung, API, Worker und das ausdrückliche Wartungsende. Der Worker teilt den PID-Namensraum der API für den bestehenden Eigentumsnachweis, besitzt aber kein Netzwerk. Ein gemeinsamer PID-Namensraum und dieselbe UID schützen nicht vor einem bösartigen gleichberechtigten Prozess; die Trennung ist kein pauschales Sicherheitsversprechen.

Nach vollständig vorbereiteter Konfiguration zeigt die Originalanleitung diese Befehle:

```
docker compose -f Slicer-Server/compose.candidate.yaml config --quiet
docker compose -f Slicer-Server/compose.candidate.yaml up -d
docker compose -f Slicer-Server/compose.candidate.yaml ps
```

Sie sind für den isolierten Kandidatenweg beschrieben. Sie erzeugen keine veröffentlichte Komplettlaufzeit. Insbesondere darf ein leeres Engine-Verzeichnis nur für entsprechend begrenzte Leerlaufprüfungen verwendet werden. Es führt nicht zu einem slicefähigen System.

Ein Dienst auf `127.0.0.1` ist nur vom eigenen Host erreichbar. Für Home Assistant auf einem anderen Host müssen Bind-Adresse, Firewall und Zertifikatsvertrauen zusammenpassen. Die Lösung ist eine bewusst eingerichtete HTTPS-Verbindung, nicht das Abschalten der Zertifikatsprüfung oder ein ungeschützter öffentlicher Port.

6. Engines und Profile passend zusammenführen

Der Server sieht drei Enginefamilien vor: Bambu Studio, PrusaSlicer und CuraEngine. Jede besitzt eigene Programme, Ressourcen und Profilformate. „Die Engine-Datei ist vorhanden“ reicht deshalb nicht aus. Ein Programm kann starten und trotzdem notwendige Definitionen, Bibliotheken oder dynamisch geladene Bestandteile vermissen.

Im geprüften `engine_catalog.json` stehen noch keine freigegebenen Releases. Alle drei Familien sind `pending`. Die vorhandenen Bundle-Referenzen dokumentieren konkrete Archive und Prüfungen, aber keine freigegebenen Runtime-Downloads. Kopiere deshalb keine Versionsnummer aus einem Nachweis in eine selbst erfundene Downloadadresse.

Der Container-Pfadvertrag ist festgelegt:

Engine-ID	Programmeinstieg im Container	Einordnung
`bambu_studio`	`/engine/bambu_studio/bin/bambu-studio`	Zusammen mit den passenden Bambu-Ressourcen
`prusaslicer`	`/engine/prusaslicer/bin/prusa-slicer`	Mit vollständigem Prusa-Ressourcen-/Bibliotheksumfang
`curaengine`	`/engine/curaengine/bin/CuraEngine`	Mit passenden Definitionen und Vererbungen

`SLICER_ENGINE` bezeichnet den gemeinsamen Stamm, nicht direkt ein einzelnes Bambu-Extraktionsverzeichnis. Die Erkennung fällt bei fehlender Engine nicht auf irgendein Programm im Host-`PATH` zurück. Eine gemeldete Version `unknown` ist ehrlicher als eine nicht nachgewiesene Versionsangabe.

Die Offline-Distributionswerkzeuge können aus tatsächlichen geprüften Eingaben einen gemeinsamen Bestand zusammenstellen. Sie erwarten alle drei Familien, die referenzierten Archive, festgelegte Größen und Prüfsummen sowie ein neues Ausgabeziel. Ein noch `pending` enthaltender Katalog wird abgewiesen. Das Werkzeug lädt fehlende Dateien nicht ersatzweise herunter und aktiviert keine laufende Installation.

Was ein Profil für deinen Auftrag bedeutet

Drucker, Düse, Platte, Filament und Prozess beschreiben unterschiedliche Eigenschaften. Der Drucker legt beispielsweise die Maschine fest; die Düse beeinflusst die zulässige Extrusion; Material und Prozess bestimmen weitere Slicing-Parameter. Eine passende Bezeichnung im Studio bestätigt nicht automatisch, dass alle Werte für jede Engine und Hardwarekombination geprüft sind.

Der dokumentierte Container-Seed enthält derzeit Drucker-JSONs für **Bambu Lab A1 und H2S mit 0,4-mm-Düse**. Daraus darf keine vollständige Freigabe weiterer Düsendurchmesser oder aller Druckermodelle abgeleitet werden. Zusätzliche gültige Druckerprofile sind im Speichervertrag möglich; ihre fachliche Eignung muss unabhängig stimmen.

Der Profilinitialisierer übernimmt geprüfte Seed-Dateien einmalig in die dauerhafte Ablage. Ein fertiger gültiger Bestand wird wiederverwendet. Änderungen an gültigen Benutzerprofilen bleiben erhalten; ein neues Image überschreibt sie nicht automatisch mit seinem Seed. Fehlt ein erforderliches Seedprofil

oder ist der Initialisierungsbeleg unklar, wird der Zustand nicht still übernommen.

Die Verbraucher binden den Profilstamm nur lesend ein. Eine spätere Profiländerung braucht deshalb einen dafür vorgesehenen kontrollierten Schreibweg. Direktes Bearbeiten unter einem laufenden Worker ist kein Ersatz für eine abgestimmte Profilverwaltung. Für zusätzliche JSON-Dateien gelten außerdem Größen-, Mengen- und Pfadprüfungen; diese technische Akzeptanz ist keine Druckqualitätsbewertung.

Materialzuordnung bewusst prüfen

Vor dem Slicing muss die Materialauswahl zum vorgesehenen Auftrag passen. Kontrolliere die ausgewählte Maschine und Düse, das Materialprofil, den Prozess und gegebenenfalls die Zuordnung mehrerer Materialien im Studio. Eine Farbdarstellung und ein Materialprofil erfüllen unterschiedliche Aufgaben: Eine Farbe ersetzt keine Temperatur- oder Flusseinstellung.

Dieses Handbuch bestätigt keine automatische Zuordnung beliebiger AMS-Slots, externer Spulen oder Fremdprofile. Die Slicerunterlagen liefern dafür keine vollständige Endnutzerabnahme. Wenn eine Kombination nicht eindeutig belegt ist, halte sie als offenen Punkt fest, statt einen ähnlich benannten Datensatz als gleichwertig anzunehmen.

7. Home Assistant verbinden

Für einen bereits eingerichteten externen Server beschreibt der native Paketweg den Einstieg **Einstellungen → Geräte & Dienste → Ultimate 3D Studio → Konfigurieren**. Die genaue verfügbare Dialogausprägung richtet sich nach der installierten Integration. Wähle dort bewusst den externen Verbindungsweg und hinterlege die tatsächliche HTTPS-Adresse sowie das zugehörige Secret.

Prüfe zuerst den Hostnamen. Er muss zum Zertifikat passen und von Home Assistant aufgelöst werden können. Danach muss Home Assistant der ausstellenden CA vertrauen. Erst wenn diese Identität stimmt, ist eine Authentifizierungsprüfung sinnvoll. Ein pauschales „Zertifikat ignorieren“ gehört nicht zum beschriebenen Einrichtungsweg.

Der Verbindungstest nutzt die geschützte Info-Schnittstelle. Er startet weder Slicing noch einen Druckerupload. Übernimm die Verbindung erst, wenn Zertifikat, Authentifizierung und API-Kompatibilität zusammenpassen. Ein offener TCP-Port oder eine erreichbare Webseite ist dafür allein zu wenig.

Bei einem Serverwechsel gehören auch der Verbindungskontext und seine alten Antworten auf den Prüfstand. Die Integrationsbeschreibung verlangt, vorherige Cacheeinträge und ausstehende Antworten nicht dem neuen Server zuzuordnen. Wenn du den Server gewechselt hast, prüfe deshalb den angezeigten neuen Stand, bevor du vorhandene Aufträge weiterverwendest.

Supervisor-App: anderer Ablauf, andere Grenzen

Die Supervisor-Verwaltung prüft zunächst App, Paketidentität und Berechtigungen. Vorgänge benötigen einen aktiven Administrator; diese Identität stammt aus dem authentifizierten Home-Assistant-Kontext. Die Prüfung akzeptiert nicht einfach eine Administratorbehauptung aus einem Formularfeld.

Eine Installation wird erst angeboten, wenn eine passende freigegebene Paketreferenz hinterlegt ist. Im geprüften Stand fehlt sie. Ein Prüfimage ohne Engines ist kein Ersatz. Die Verwaltung startet, aktualisiert oder entfernt im beschriebenen Adaptervertrag keine App; Installation und Betriebsstart sind ausdrücklich getrennt.

Bei einer Unterbrechung bleibt eine gespeicherte Vorgangsabsicht erhalten. Ein Timeout ist keine Erlaubnis für eine zweite Installation. Ein vorhandener laufender Container kann als `running_unverified` erscheinen. Das bestätigt weder den Zugangsschlüssel noch eine erfolgreiche Slicing-Verbindung. Ein bestätigter Bestandsabgleich zu `reconciled_stopped` betrifft die lokale Quittung und startet ebenfalls keinen Dienst.

8. Einen Slicing-Ablauf nachvollziehbar durchführen

Der folgende Ablauf beschreibt die Bedienreihenfolge für einen später vollständig eingerichteten und freigegebenen Dienst. Er ist kein Protokoll eines bereits bestandenen echten Slices. Der gegenwärtig fehlende Runtime-Download bleibt eine reale Grenze.

1. **Modell und Ziel festlegen.** Öffne das Modell im Studio und prüfe Maße, Ausrichtung und die vorgesehene Platte. Verwende für eine erste unabhängige Prüfung ein eigenes überschaubares Modell, dessen erwartete Eigenschaften du kennst.
2. **Maschine und Prozess prüfen.** Vergleiche Drucker, Düse, Material und Prozess mit dem tatsächlich vorgesehenen Bestand. Ein häufiger Wechsel von Engine oder Profilen erschwert später die Ursachenanalyse.
3. **Verbindung kontrollieren.** Prüfe den gewählten Server und dessen Zustand. `degraded` ohne Engine ist ein erklärbarer eingeschränkter Zustand, aber keine Slicing-Bereitschaft.
4. **Den Auftrag bewusst anlegen.** Slicing-Auftrag, Modellübertragung und Druckstart sind getrennte Aktionen. Führe nur den beabsichtigten Schritt aus und beobachte dessen Rückmeldung.
5. **Verarbeitung verfolgen.** Halte fest, welcher Auftrag mit welcher Auswahl verarbeitet wird. Nach einem Timeout zunächst den vorhandenen Zustand prüfen; nicht automatisch einen zweiten gleichartigen Auftrag erzeugen.
6. **Ergebnis überprüfen.** Prüfe im Studio die tatsächlich zu diesem Auftrag gehörenden Ergebnisse und die Vorschau. Eine berechnete Datei ist noch kein Nachweis eines erfolgreichen physischen Drucks.
7. **Erst danach weiterentscheiden.** Ein späterer Upload oder Druckstart braucht seine eigene bewusste Aktion und die dafür vorgesehenen Kontrollen.

Aufträge, die auf Verarbeitung warten, sind ebenfalls Teil des Bestands. Nach einem Neustart darf ein alter freigegebener Auftrag nicht unbemerkt unter einer anderen Enginegeneration ausgeführt werden. Notiere deshalb bei Wartung auch wartende Einträge und nicht nur den gerade sichtbaren laufenden Job.

Ein Abbruch soll über die authentifizierte API beziehungsweise die dafür vorgesehene Bedienfunktion erfolgen. Der zugehörige Supervisor beendet seine eigene Prozessgruppe. Teilweise erzeugte Druckartefakte eines abgebrochenen Auftrags dürfen nicht als fertiges Ergebnis erscheinen. Das manuelle Beenden irgendeines gleichnamigen Prozesses liefert keinen sauberen Auftragsabschluss.

Bei einem Host- oder Containerausfall können verwaiste `slicing`-Marker oder eine Recovery-Sperre erhalten bleiben. Das ist ein Hinweis auf notwendigen Abgleich. Die richtige Reaktion lautet: Dienste kontrolliert stoppen, Zustand und Daten sichern und die betroffene Verarbeitung untersuchen. Die bloße Existenz einer Ergebnisdatei ersetzt diesen Abgleich nicht.

9. Authentifizierung, TLS und Datengrenzen

Die öffentliche Health-Schnittstelle und die geschützte Info-Schnittstelle haben verschiedene Zwecke. `GET /api/v1/health` kann Erreichbarkeit und einen eingeschränkten Zustand melden. `GET /api/v1/info` benötigt `Authorization: Bearer <Secret>`. Ein Health-Erfolg sagt deshalb nichts darüber aus, ob das Secret korrekt ist.

Das Secret bleibt in der dafür vorgesehenen privaten Ablage. Auch scheinbar harmlose Diagnosekopien können Konfiguration oder private Modelle enthalten. Prüfe vor dem Teilen von Logs, Screenshots und Archiven, ob darin Header, Schlüssel, Verbindungsdaten oder Nutzerdateien vorkommen. Das Handbuch benötigt keinen echten Secret-Wert als Beispiel.

TLS besteht aus mehreren zusammengehörigen Eigenschaften: passender Hostname, gültiges Zertifikat, vertrauende Gegenstelle und geschützter privater Schlüssel. Ein lesbares Zertifikat allein stellt keine vertrauenswürdige Verbindung her. Der native Paketweg erlaubt eine andere Bind-Adresse als Loopback nur mit vollständiger TLS-Konfiguration; alternativ ist ein entsprechend eingerichteter lokaler Reverse-Proxy beschrieben.

Die Container laufen ohne Root, zusätzliche Capabilities und Docker-Socket. Das Root-Dateisystem und der Enginebestand sind nur lesbar eingebunden. Diese Eigenschaften begrenzen Eingriffe, ersetzen aber keine Prüfung des konkreten Hosts, der verwendeten Images und der tatsächlichen Mounts. Ein privilegierter Hostadministrator kann weiterhin Daten außerhalb des Containers verändern.

Programmcode, Engine und Nutzdaten sollten sich auch in deiner mentalen Ordnung unterscheiden. Ein neues Image soll keine privaten Modelle enthalten. Eine Sicherung von Nutzdaten kann dagegen sehr persönliche Inhalte enthalten. Konfiguration und Schlüssel benötigen eine andere Weitergabebehandlung als öffentlich lesbare Programmquellen.

Lizenzgrenzen gehören zur Paketidentität

Das Repository enthält Komponenten mit unterschiedlichen Rechten. Bereits wirksam unter MIT veröffentlichte Teile behalten ihre Hinweise. Für neues eigenes Material im abgegrenzten Slicerbereich gilt ein eigener Source-available-Lizenztext. Übernommene Engines, Bibliotheken, Ressourcen und Profile behalten ihre jeweils anwendbaren Bedingungen.

Deshalb beschreibt dieses Handbuch weder das gesamte Repository als MIT-lizenziert noch ein künftiges Containerimage als ausschließlich von der eigenen Lizenz erfasst. Ein Hash bestätigt Identität, keine Rechtsfreigabe. Ebenso macht ein beigefügter AGPL-Text aus einem unvollständigen Enginepaket noch keine abgeschlossene korrespondierende Quellenauslieferung.

Maßgeblich sind Repository-Hinweise, Slicer-Lizenz, Drittanbieterhinweise und die tatsächlichen Paketbestandteile. Diese Texte werden durch den Handbuchentwurf nicht verändert. Vor einer Engineweitergabe müssen die konkreten Herkunfts- und Auslieferungspflichten des betreffenden Bestands geklärt sein.

10. Status beobachten und Fehler früh einordnen

Für den Compose-Kandidaten dokumentiert die Laufzeitanleitung:

```
docker compose -f Slicer-Server/compose.candidate.yaml logs --tail 100 api worker
docker compose -f Slicer-Server/compose.candidate.yaml exec worker \
  python3 /opt/ultimate-slicer-server/container_service.py worker-check
```

Die Logansicht hilft, API- und Workerzustand zusammen zu betrachten. Der Worker-Check bezieht Herzschlag, Prozessidentität und Recovery-Sperre ein. Er ist kein Ersatz für eine vollständige Engineprüfung. Behandle die Ausgabe als Diagnoseinformation zum jeweiligen Dienststand und bewahre sie nicht ungeprüft öffentlich auf.

Bei einer Störung lohnt sich ein schrittweises Vorgehen: Ist der richtige Dienst erreichbar? Passt sein Zertifikat? Wird die Authentifizierung akzeptiert? Meldet er die erwartete API-Kompatibilität? Ist die Engine vollständig vorhanden? Erst danach werden auftrags- oder profilspezifische Ursachen sinnvoll unterscheidbar.

Eine hilfreiche Fehlermeldung nennt Softwarestand, Architektur, betroffene Funktion, Zeitpunkt und einen bereinigten Fehlercode. Bei einem konkreten Auftrag gehört eine nachvollziehbare Zuordnung dazu. Für die erste Diagnose ist üblicherweise nicht dein gesamtes Datenverzeichnis nötig. Private Modelle, vollständige Konfigurationen und Rohbackups gehören nicht ungeprüft in ein öffentliches Issue.

Automatische Neustartschleifen sind im Kandidaten ausgeschaltet. Ein ungesunder Dienst wird daher nicht still im Hintergrund „repariert“. Das erleichtert die Zustandsanalyse, verlangt aber einen bewussten Wiederanlauf. Wiederholtes Starten ohne Klärung einer erhaltenen Recovery-Sperre führt nicht zu einem sauberen Ergebnis.

11. Wartung, Sicherung und Wiederherstellung

Eine Wartung soll einen nachvollziehbaren Ausgangs- und Zielstand besitzen. Halte Anwendung beziehungsweise Image-ID, Enginegeneration und Konfiguration fest. Prüfe vor Änderungen, ob Aufträge laufen oder noch zur Verarbeitung freigegeben sind. Ein Update unter laufender Berechnung ist kein geeigneter Normalweg.

Der vorhandene Wartungsvertrag verhindert neue Jobstarts, beendet aber keinen aktiven Auftrag. Der Zustand liegt dauerhaft im Datenbereich und überlebt Neustarts. Ein gesund pausierter Worker ist daher möglich. Die Wartungskennung gehört zum eigenen Vorgang; sie ist kein allgemeines Authentifizierungstoken und darf nicht zur Übernahme einer fremden Sperre dienen.

Ein Timeout beim Setzen einer Wartungssperre kann nach einer bereits erfolgten Änderung auftreten. Prüfe deshalb den tatsächlichen Zustand, bevor du fortsetzt. Eine fehlende Antwort beweist weder Erfolg noch Wirkungslosigkeit. Die dokumentierte Rücknahme benötigt die passende Vorgangsidentität und erneute Zustandsprüfungen.

Was in eine Sicherung gehört

Bestand	Warum er gebraucht wird	Besonderheit
Konfiguration und Verbindungsdaten	Den Dienst identisch einrichten	Kann Secrets enthalten
TLS-Dateien und API-Token	Die Verbindung wiederherstellen	Separat geschützt aufbewahren
Eigene Profile	Die beabsichtigten Parameter erhalten	Nicht mit Image-Seeds verwechseln
Jobs, Uploads und Ergebnisse	Verlauf und Datenbestand erhalten	Kann private Modelle und G-Code enthalten
Image-ID und Enginegeneration	Den passenden Programmstand wählen	Zusammengehörigkeit dokumentieren
Betriebs-/Compose-Konfiguration	Mounts und Laufzeit nachvollziehen	Keine privaten Werte veröffentlichen

Vor einer konsistenten Sicherung oder einem Restore werden beide Dienste gestoppt und ihr Stillstand geprüft. Eine Kopie während unklarer Schreibvorgänge ist nicht automatisch ein wiederherstellbarer Stand. Die Sicherung wird anschließend zurückgelesen und auf Vollständigkeit geprüft. Erst dadurch entsteht ein belastbarer Rückweg.

Einen Restore zuerst in einem neuen privaten Testpfad vorzunehmen, erhält den Ausgangsbestand. Bewahre Eigentümer, Gruppen und erforderliche Rechte und vergleiche die Dateihashes. Ein leeres Testverzeichnis darf niemals versehentlich als Ersatz für ein produktives Volume eingesetzt werden.

Container stoppen und neu starten

```
docker compose -f Slicer-Server/compose.candidate.yaml stop
docker compose -f Slicer-Server/compose.candidate.yaml down
docker compose -f Slicer-Server/compose.candidate.yaml up -d
```

Diese Befehle sind dokumentierte Bestandteile des Kandidatenwegs, keine automatische Updateprozedur. Bei einem Imagewechsel oder Anchor-Neustart müssen API und Worker gemeinsam zum passenden Stand gehören. Ein Worker soll nicht im alten PID-Namensraum verbleiben, während die API bereits ersetzt wurde.

Ein Image-Rollback und ein Datenrestore sind verschiedene Maßnahmen. Eine ältere Anwendung darf einen eventuell veränderten neueren Datenbestand nicht ungeprüft übernehmen. Ebenso setzt ein Imagewechsel eine vorhandene Recovery-Markierung nicht zurück. Entscheidend ist, dass Anwendung, Engine, Profile und Datenformat wieder zusammenpassen.

Der native Installer kennt zusätzlich einen begrenzten Code-only-Upgrade-/Rollbackvertrag. Er ändert dabei nicht beliebig systemd-Definitionen oder Datenformate. Dieser Vertrag ist kein allgemeiner Migrationsassistent und im geprüften Entwicklungsstand keine Installationsfreigabe. Native Dienste und Compose dürfen denselben Bestand nicht versehentlich parallel verarbeiten.

12. Fehlersuche: Symptom, Prüfung, Maßnahme

Die Tabelle nennt belegte Zustände und sinnvolle Prüfreihenfolgen. Sie ersetzt keine Untersuchung des tatsächlichen Datenbestands. Bei widersprüchlichen oder unbekannten Zuständen wird die Verarbeitung angehalten, statt den vorhandenen Beleg passend zu machen.

Symptom	Zuerst prüfen	Maßnahme
Home Assistant erreicht den Dienst nicht	URL, Hostnamenauflösung, Bind-Adresse und Firewall	Tatsächlichen Netzwerkpfad klären; Loopback ist von einem anderen Host nicht erreichbar
TLS-Prüfung scheitert	Zertifikatsname, Gültigkeit, CA-Vertrauen und Serverkonfiguration	Richtige Identität und Vertrauenskette herstellen; Prüfung nicht abschalten
Health antwortet, Verbindung bleibt ungültig	Geschützte Info-API, Secret und API-Kompatibilität	Die Authentifizierung getrennt von Erreichbarkeit prüfen
Health meldet `degraded`	Enginebestand und erwartete Programmeinstiege	Fehlenden oder unvollständigen Enginebestand klären; keinen Drucknachweis daraus ableiten
Keine Engineversion bekannt	Tatsächlich vorhandene Versionsmetadaten	`unknown` erhalten, bis eine geprüfte Zuordnung vorliegt
`source_digest_mismatch`	Klon, Liefermanifest und tatsächlich veränderte Datei	Ursache und maßgeblichen Quellstand ermitteln; Hash nicht blind austauschen
`output_already_exists`	Bereits verwendetes Ausgabeverzeichnis	Neues eigenes Ziel wählen oder den eigenen alten Kontext bewusst verwalten
`unexpected_context_file`	Zusätzliche Datei im Build-Kontext	Kontext aus erlaubten Quellen neu erzeugen; Secrets und Nutzdaten draußen halten
Profilinitialisierung blockiert	Profilstamm, erforderliche Seeds und `PROFILE_STATE.json`	Verbraucher stoppen und Zustand mit gesicherten Originalen abgleichen
Worker startet keine weiteren Aufträge	Wartungs-/Recovery-Zustand und `*.slicing.json`	Betroffenen Vorgang untersuchen; Marker nicht automatisch löschen
Supervisor-Vorgang bleibt `pending`	Gespeicherte Absicht und tatsächlicher Appzustand	Bestandsabgleich verwenden, keinen blinden zweiten Installationsversuch starten
App ist `running_unverified`	Appidentität und separate Slicer-Verbindung	Laufzustand nicht als gültige Zugangskonfiguration ausgeben
Dateienzugriff scheitert	Gemounteter Pfad, Eigentümer, Gruppe und Rechtevertrag	Ursache am betroffenen Bereich beheben; keine pauschale rekursive Rechteänderung
Neustart zeigt alte Aufträge	Datenpfad, Queue und verwendete Generation	Bestand bewusst abgleichen, bevor Verarbeitung wieder freigegeben wird

Wenn ein Problem nach einem Update beginnt, halte zuerst die beteiligten Versionsstände fest. Der Zusammenhang „neues Image, neuer Fehler“ ist ein guter Ausgangspunkt, aber noch keine genaue Ursache. Prüfe auch, ob Mounts, Rechte oder die Enginegeneration mitgewechselt wurden. Ein vollständiger Rückweg erhält diese Informationen.

Bei einer abgebrochenen Installation gilt besondere Sorgfalt: Eine vorbereitete Datei oder ein vorhandenes Verzeichnis ist keine Fertigmeldung. Der übergeordnete Vertrag muss den erreichten Zustand bestätigen. Manuelles Löschen von Journalen kann genau die Information entfernen, die für einen sicheren Bestandsabgleich benötigt wird.

13. Häufige Fragen

Kann ich den Slicer nach dem Klonen sofort verwenden?

Du kannst die veröffentlichten Quellen und Prüfwerkzeuge nachvollziehen. Ein vollständiger installierbarer Runtime-Download mit allen vorgesehenen Engines ist noch nicht veröffentlicht. Für echte Verarbeitung werden ein vollständig eingerichteter Dienst und passende geprüfte Ressourcen benötigt.

Installiert HACS den Linux-Server mit?

Nein. Das native Paket-README trennt die Home-Assistant-Integration vom unabhängigen Linux-Slicer. Auch eine verfügbare Integration ersetzt nicht die App-Paketreferenz oder eine externe Serverinstallation.

Warum antwortet Health, obwohl Slicing nicht möglich ist?

Erreichbarkeit, Authentifizierung und Enginebereitschaft sind unterschiedliche Eigenschaften. Ein leerer Engine-Speicher kann zu `degraded` führen, während API und Worker ihren begrenzten Leerlaufvertrag erfüllen.

Darf ich einen fehlenden Enginepfad auf eine andere installierte Engine umbiegen?

Der Container verwendet feste Engine-IDs und Einstiegspfade. Ein Ersatzprogramm im Host-PATH ist nicht der dokumentierte Vertrag. Unterschiedliche Enginefamilien benötigen passende Ressourcen und Profilformate.

Werden meine Profile beim Imagewechsel überschrieben?

Der beschriebene Profilvertrag erhält gültige bestehende Benutzerprofile und überschreibt sie nicht automatisch mit einem neuen Seed. Voraussetzung ist ein gültiger vorbereiteter Bestand; unklare oder unvollständige Ablagen werden nicht einfach übernommen.

Ist ein erfolgreicher Test ein Release?

Nein. Quelltests, Paketidentität, Imagebau, Enginefunktion und echte Neuinstallation belegen verschiedene Umfänge. Dieses Handbuch setzt `release_complete`, `production_approved` oder andere Freigabefelder nicht auf wahr.

Kann ich das System beruflich oder verändert weitergeben?

Die Rechte unterscheiden sich nach Komponente. Für neues eigenes Material nennt der abgegrenzte Slicer-Lizenztext Beschränkungen; zuvor gewährte und fremde Rechte bleiben bestehen. Maßgeblich sind die konkreten Lizenztexte, nicht eine vereinfachte Aussage dieses Handbuchs.

Warum zeigt die Anleitung keine fertige öffentliche Imageadresse?

Im geprüften Stand existiert keine freigegebene veröffentlichte Referenz für die vollständige Runtime. Eine erfundene Registryadresse oder ein bewegliches Tag würde die fehlende Auslieferung nur verdecken.

Kann ich einfach noch einmal installieren, wenn ein Vorgang hängt?

Bei gespeicherter Vorgangsabsicht muss zuerst der tatsächliche Bestand abgeglichen werden. Ein Timeout kann nach einer bereits erfolgten Änderung eintreten. Wiederholung ist daher keine allgemeine Fehlerbehandlung.

Kann ich Modelle während einer Wartung bereits weiter vorbereiten?

Die Integrationsbeschreibung sieht vor, dass Galerie, CAD und Profilbearbeitung nicht von einer vorübergehenden Slicerstörung abhängen sollen. Ob ein konkreter vorbereiteter Auftrag anschließend verarbeitet werden darf, wird getrennt am Serverzustand und der gewählten Konfiguration entschieden.

14. Begriffe und nächste Orientierung

Begriff	Bedeutung in diesem Projekt
API	Schnittstelle zwischen Home Assistant und Slicing-Dienst
Worker	Verarbeitungskomponente für Aufträge
Engine	Programm für die eigentliche Berechnung der Druckbahnen
Profil	Parameterbestand für Maschine, Material, Prozess oder zugehörige Eigenschaften
Enginegeneration	Zusammengehöriger geprüfter Bestand mehrerer Engine-Dateien und Metadaten
Image	Anwendungs-/Systemdateien eines Containerstands
Container	Laufende Instanz eines Images mit den vorgesehenen Mounts und Rechten
Mount	Zuordnung eines Hostbereichs in den Container
Digest / SHA-256	Prüfsumme zur Identifikation konkreter Bytes
Preflight	Vorprüfung ohne die vollständige eigentliche Betriebsaktion
Pairing	Ausdrückliche Einrichtung einer Vertrauensbeziehung zum Hostkanal
Wartungsbarriere	Dauerhafte Sperre neuer Jobstarts während eines kontrollierten Wechsels
Recovery	Erforderlicher Abgleich nach unklarem oder unterbrochenem Zustand
Supervisor-App	Eigenständiger durch Home Assistant Supervisor verwalteter Anwendungsweg
HACS	Auslieferungsweg der HA-Integration; kein allgemeiner Linux-Engineinstaller

Für den praktischen Einstieg lies als Nächstes Installation. Für den externen Kandidatenweg gehören Docker-Schnellstart und Laufzeitkonfiguration zusammen. Wer den Betrieb plant, ergänzt Betrieb und Migration. Die Enginebestandteile erklärt Engine-Herkunft.

Offen bleiben der vollständige Engine-/App-Download, konkrete Freigaben und reale unabhängige Abnahmen. Auch dieses ausführliche Handbuch ist keine Funktionsabnahme der Studio-Gesamtmatrix. **Der Gesamtproduktrelease bleibt offen; die eigenständige Linux-Neuinstallation wird zuletzt verifiziert.**